
— AI SECURITY · AGENTIC SYSTEMS

Securing Agentic AI

Governing what the system does, **not what it says.**

A reference architecture for autonomous agents in regulated environments: enforcement at the tool boundary, bounded blast radius, and authorization that stays current with the system it authorizes.

TOOL-BOUNDARY ENFORCEMENT / BLAST RADIUS / CONTINUOUS ATO / EVAL GATES

Ryan Gutwein

Founder & CEO, Optimal, LLC · CISSP · CCSP · CCP

goptimal.io · ryan@goptimal.io

Sovereign by default · CUI & ITAR-ready · SDVOSB-certified · CAGE 14HQ0 · UEI TYMSGKDF2K48

© 2026 Optimal, LLC

Contents

Executive Summary

1. Why Existing Models Fall Short

2. Reference Architecture

3. Threat Model

4. Design Principles

5. Observability

6. Registry and Supply Chain Controls

7. Evaluation and Assurance

8. Governance

9. Control Crosswalk

10. Implementation Roadmap

Conclusion

References

Executive Summary

Enterprise AI has crossed a threshold. For three years the dominant deployment pattern was conversational: a model produced text and a human decided what to do with it. The security question was **content risk**. Agents built on the Model Context Protocol (MCP) and comparable tool-calling frameworks now invoke APIs, mutate database state, provision infrastructure, move money, and call third-party services without a human in the path. The governing question has shifted from *what the AI says* to **what the AI does**. That is **action risk**, and it is a different discipline.

The timing makes the discipline urgent. Offense operates at machine speed: autonomous tooling finds and exploits flaws faster than human review cycles can close them, while an agent's own behavior changes with every model version, tool manifest update, and memory write, faster than any annual assessment can observe. Organizations that respond by refusing agents are not holding a safe position; they are accepting a different, unsigned risk. And because the dominant attack class against agents has no complete mitigation, prevention-first strategies fail by construction. Survivability must be engineered: bounded blast radius, enforcement the compromised component cannot reach, and authorization that stays current with the system it authorizes.

Three properties make action risk resistant to existing controls:

1. **Non-determinism.** An agent may reach the same goal through different plans on different runs. Perimeter rules written as static if/then logic cannot constrain a system that composes its own execution path.
2. **Semantic attack surface.** The primary vulnerability is not in code but in reasoning. An agent that ingests untrusted content (a support ticket, a PDF, a webhook payload, a retrieved document) can have its objective rewritten by text inside that content. This is not a memory-safety bug that can be patched.
3. **Delegated privilege.** Agents are useful in proportion to their access. An agent granted broad standing permissions becomes a *proxy insider*, an always-on, highly privileged actor whose judgment can be influenced by anyone who can get text in front of it.

This paper argues that securing agentic systems requires moving enforcement out of the model and into the **execution path**. Specifically:

- **Authorization must be enforced at the tool boundary**, not inside the agent runtime. Any control that lives in the same trust domain as the reasoning it constrains is advisory, not enforcing.
- **Containment and authorization are separate blast radii.** Sandboxing bounds host compromise. It does nothing to bound the damage of an action the agent was permitted to take. Most published architectures conflate these.
- **Delegation semantics are the unsolved core problem** in multi-agent systems. Without cryptographic propagation of the originating principal's identity and scope across agent hops, Confused Deputy is unmitigated regardless of registry policy.
- **Assurance requires a stated evaluation methodology with numeric gates.** Prompt injection has no complete mitigation. An architecture that recommends controls without a formal residual-risk acceptance is incomplete for any regulated audience.

Together, the promotion gates, continuous evaluation, and re-authorization triggers defined here constitute a continuous authorization (cATO) model for agentic systems: the only authorization model that matches an artifact that changes underneath its assessment. The paper closes with an evaluation framework, a governance model, and a control crosswalk to NIST SP 800-53 and ISO/IEC 42001 so the architecture can be carried into an authorization package rather than remaining a position statement.

1. Why Existing Models Fall Short

1.1 The deterministic assumption

Application security has always assumed a fixed control-flow graph. Given the code, you can enumerate the reachable states. Threat models, SAST rules, and WAF signatures all inherit this assumption.

An agent's control flow is generated at runtime by a statistical model conditioned on inputs the defender does not control. There is no complete enumeration. Two consequences follow:

- **Testing is sampling, not proof.** A single passing test run is weak evidence. Assurance requires repeated trials with reported variance (see §7).
- **Enforcement cannot rely on the plan being predictable.** It must constrain the *action*, at the moment of the action, against policy that exists outside the agent.

1.2 The two blast radii

Published guidance routinely recommends running agents in gVisor sandboxes or micro-VMs and treats this as the containment story. It is half of one.

Containment blast radius is what an attacker reaches by escaping the execution environment, the host, the container network, adjacent workloads. Sandboxing addresses this well.

Authorization blast radius is what the agent can do *through its permitted tools while working exactly as designed*. A perfectly sandboxed agent holding write credentials to a production database can still delete the production database. No amount of kernel isolation touches this.

Containment Blast Radius vs. Authorization Blast Radius

Sandboxing bounds the smaller exposure. Action risk lives almost entirely in the second.

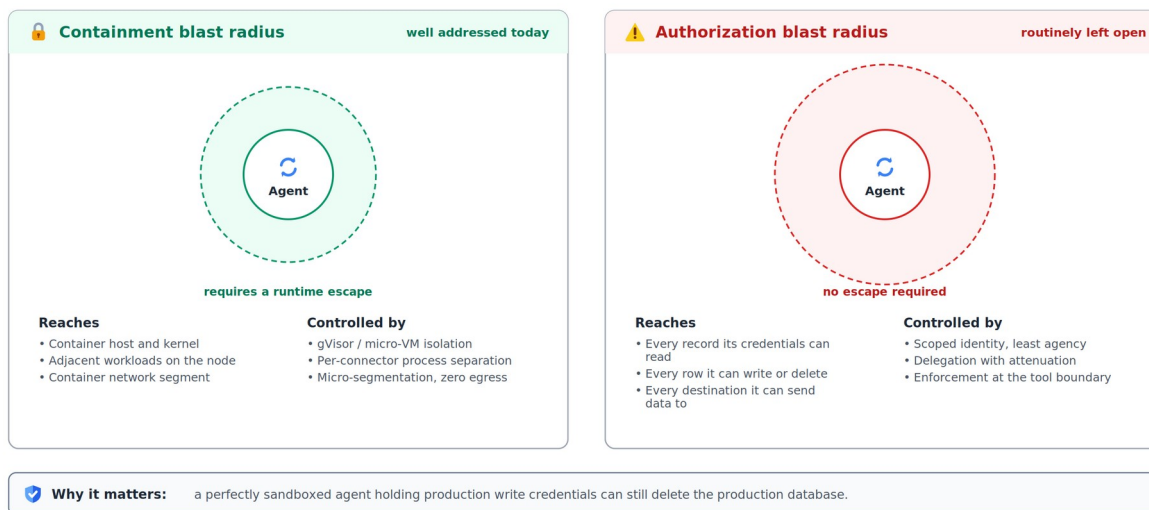


Figure 4. Containment blast radius versus authorization blast radius.

Action risk lives almost entirely in the second radius. Architectures that lead with sandboxing are optimizing the smaller exposure. **Where the agent runs** and **what the agent touches** are independent design problems and should be documented as such.

1.3 The proxy insider

Agents are granted elevated access because access is what makes them useful. The result is a non-human identity with standing privilege, no fatigue, machine-speed execution, and a decision process influenceable by any text it reads.

This is not a novel threat category, it is the insider threat with the compromise cost reduced to near zero. Where subverting a privileged human requires coercion or recruitment, subverting a privileged agent may require a well-crafted paragraph in a document it was asked to summarize.

2. Reference Architecture

Security controls must map to the operational components of an agent. Five are sufficient for most designs:

1. Reasoning engine (model host). Interprets intent, evaluates options, produces multi-step plans. Treated as **untrusted** in this architecture, it is influenced by every input it sees.

2. Memory layer. Short-term context window state and long-term persistent stores (vector indexes, conversation history, retrieved corpora). Long-term memory is a *durable* attack surface: it survives session teardown and defeats ephemerality assumptions.

3. Execution loop (harness). The orchestration code running the observe → plan → invoke → update cycle. Runs in the same trust domain as the reasoning it orchestrates. **Not a valid location for authorization enforcement.**

4. Tool layer (MCP servers, plugins, APIs). The privileged interfaces that convert reasoning into effect. This is the **policy enforcement point** and the highest-value component in the system.

5. Identity and registry layer. Signs, catalogs, and authorizes agent identities, tool manifests, and policy. The trust anchor, and therefore a concentrated target requiring its own threat model.

Core Components of an Agentic System

The reasoning engine, memory, and harness share one trust domain. Enforcement sits outside it.

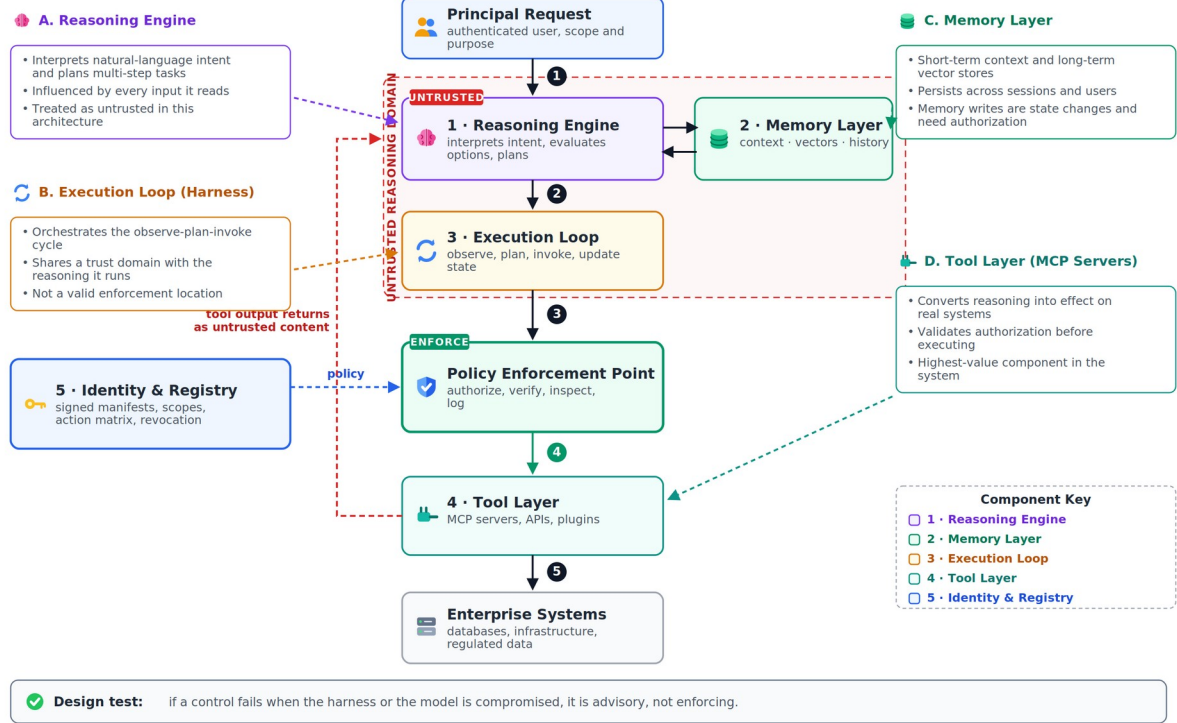


Figure 1. Agentic reference architecture and trust domains.

A useful test of any proposed control: *which of these five components would have to be compromised for the control to fail?* If the answer is “the harness” or “the model,” the control is advisory.

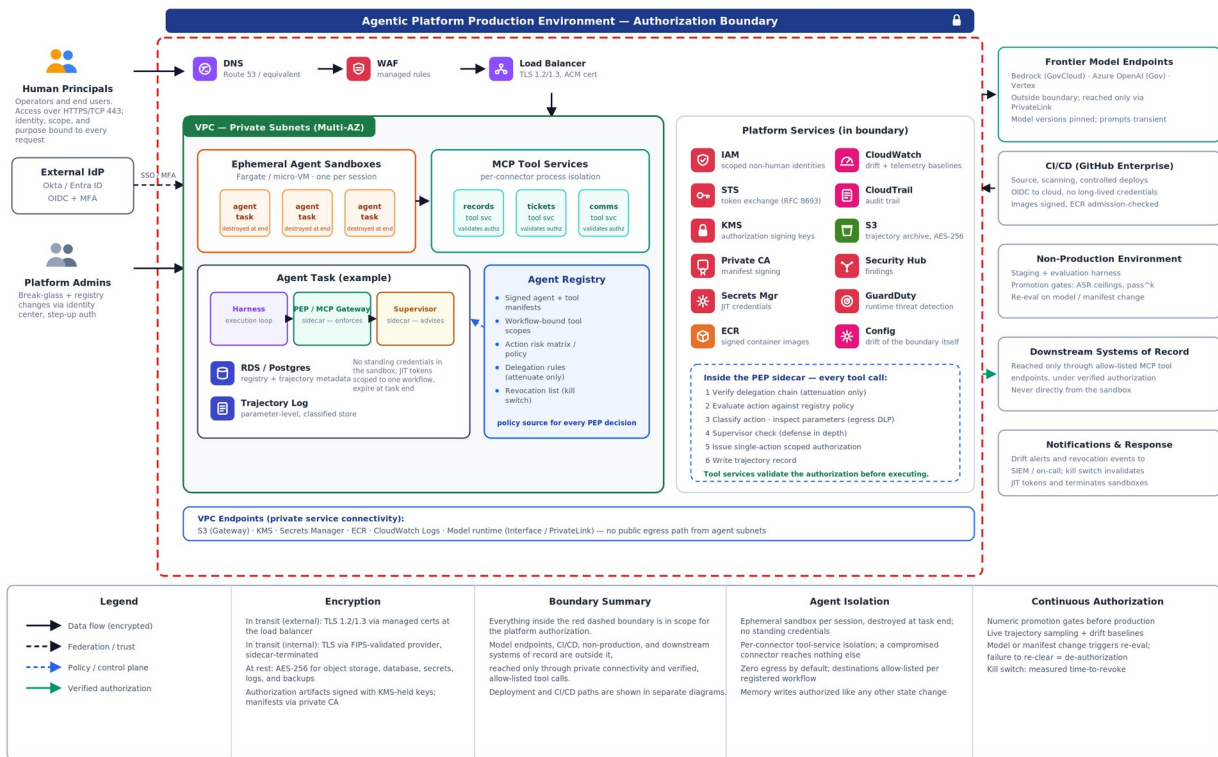


Figure 2. Reference deployment and authorization boundary. AWS GovCloud (US) shown; Azure Government and Google Cloud Assured Workloads equivalents map directly.

3. Threat Model

Agentic Threat Landscape

Each threat class targets a specific component. Controls that do not map to a component are not controls.

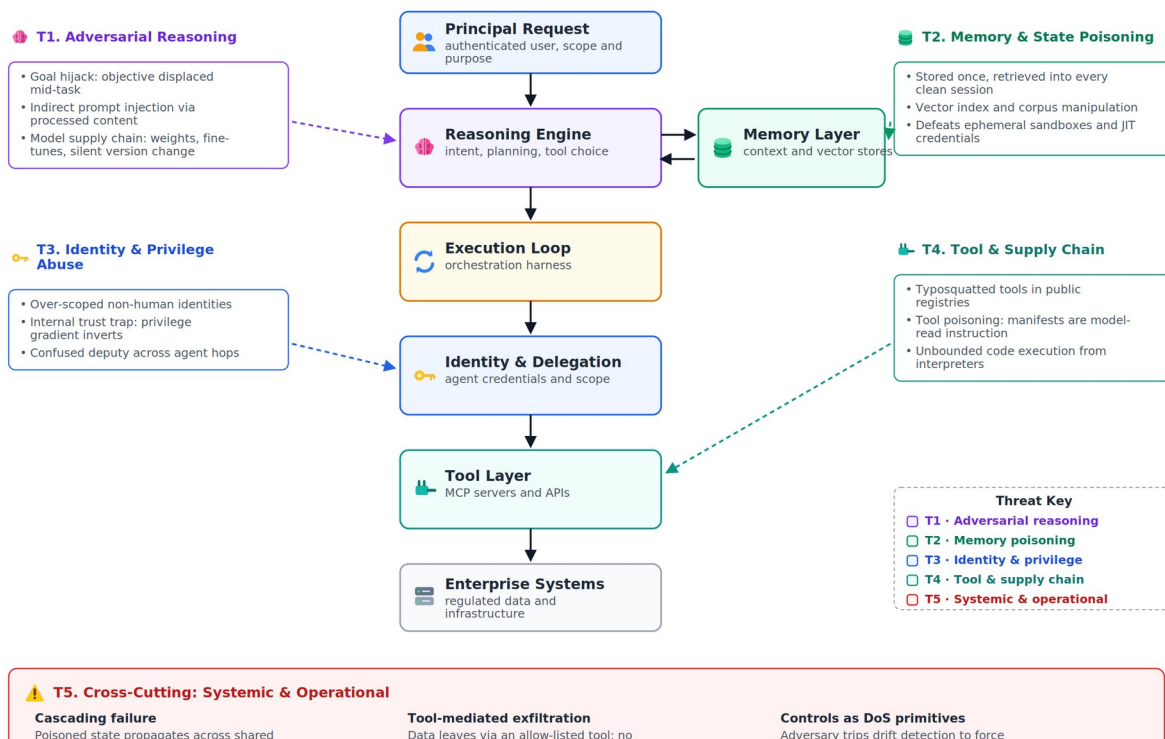


Figure 3. Threat landscape mapped to architectural components.

3.1 Adversarial reasoning

Goal hijack. An attacker manipulates context to displace the agent's registered objective. The agent continues operating normally by every conventional signal (authenticated, in-policy on the network, using approved tools) while pursuing an objective it was not deployed for.

Indirect prompt injection (IDPI). The dominant vector. The agent processes untrusted external content (documents, tickets, emails, retrieved passages, API responses, tool output) containing instructions it interprets as authoritative. Critically, IDPI does not require the attacker to interact with the system directly. Anyone who can place content where the agent will read it is in the attack surface.

No complete mitigation exists. Instruction/data separation, delimiter schemes, and classifier filters raise cost; none eliminate the class. Architectures must assume reasoning-layer compromise as a design condition, not a residual edge case.

3.2 Identity, privilege, and delegation

Over-scoped non-human identities. Agents inherit shared service accounts with standing broad privilege, typically because scoping per-workflow is operationally harder. In regulated environments this is a direct control failure against least-privilege requirements before any attack occurs.

The internal trust trap. A low-privilege agent sends a request to a high-privilege agent. If the receiving agent trusts the request because it originated inside the trust boundary, the privilege gradient is inverted, the low-

privilege agent has effectively borrowed the high-privilege agent's authority. Network-origin trust is precisely what zero trust exists to eliminate, yet most multi-agent designs reintroduce it at the agent-to-agent layer.

Confused deputy. An agent uses its own elevated permissions to perform an action the requesting principal could not perform directly. This is the general form of the above and the reason delegation semantics matter more than registry policy: policy describes what the *agent* may do, while the actual question is what this *principal, through this agent, in this context* may do.

3.3 Supply chain

Typosquatted and malicious tools. Public MCP and skill registries are unauthenticated distribution channels. A near-miss package name installed once becomes a persistent exfiltration path.

Tool poisoning via metadata. Tool descriptions are consumed by the model as natural language. An attacker who modifies a description changes what the model believes the tool does, a function documented as retrieval that performs deletion. The model is behaving correctly on false information.

Model supply chain. Weights, fine-tunes, adapters, and hosted-model version changes. A silently updated hosted model invalidates every prior evaluation result and every behavioral baseline derived from them.

3.4 Memory and state poisoning

Underweighted in most published guidance. Content written into long-term memory or a vector index persists across sessions and users. A single successful injection that causes the agent to *store* attacker-controlled content creates a durable backdoor that ephemeral sandboxes and short-lived credentials do not address, the payload is retrieved fresh into each new clean session.

Memory writes deserve the same authorization treatment as any other state-changing action.

3.5 Operational

Unexpected code execution. Agents with code interpreters can be induced to escape intended execution scope into the host or network.

Cascading failure. In interconnected multi-agent systems, a poisoned state or unhandled exception propagates at machine speed across shared workflows. Beyond cost overrun, this is an availability threat to any process the agent chain supports.

Security controls as availability attacks. An adversary who understands the goal-drift detector can deliberately trip it to force revocation of a production agent. Every automatic kill mechanism is also a denial-of-service primitive.

4. Design Principles

4.1 Least agency

Least privilege constrains **data access**. Least agency additionally constrains **functional autonomy**, the set of actions available at all.

- **Scoped, dynamic toolsets.** No standing access to the full tool catalog. Tools are bound to registered workflows and injected per session. The binding decision must come from the registry, not from the

model, otherwise tool selection reintroduces the non-deterministic surface the control was meant to remove.

- **Ephemeral credentials.** Short-lived, just-in-time tokens scoped to a single workflow and expiring at task completion. Static API keys held by agents are the single most common finding in agentic deployments.
- **Write-path minimalism.** Read and write should be distinct identities with distinct approval paths, not distinct permissions on one identity.

4.2 The enforcement point

Authorization must be verified at the tool boundary: the MCP server or an inline gateway in front of it, by a component the agent cannot influence.

This is the load-bearing claim of this architecture. A validation layer implemented inside the harness shares a trust domain with the reasoning it audits; anything that compromises the loop compromises the check. The enforcement decision must be made by a service that:

- receives the proposed action and the delegated authorization context,
- evaluates it against registry policy independent of the agent's stated justification,
- and returns a verifiable authorization that the tool endpoint **validates before execution**.

Authorization Data Flow

The action is refused by a component the compromised path cannot reach.

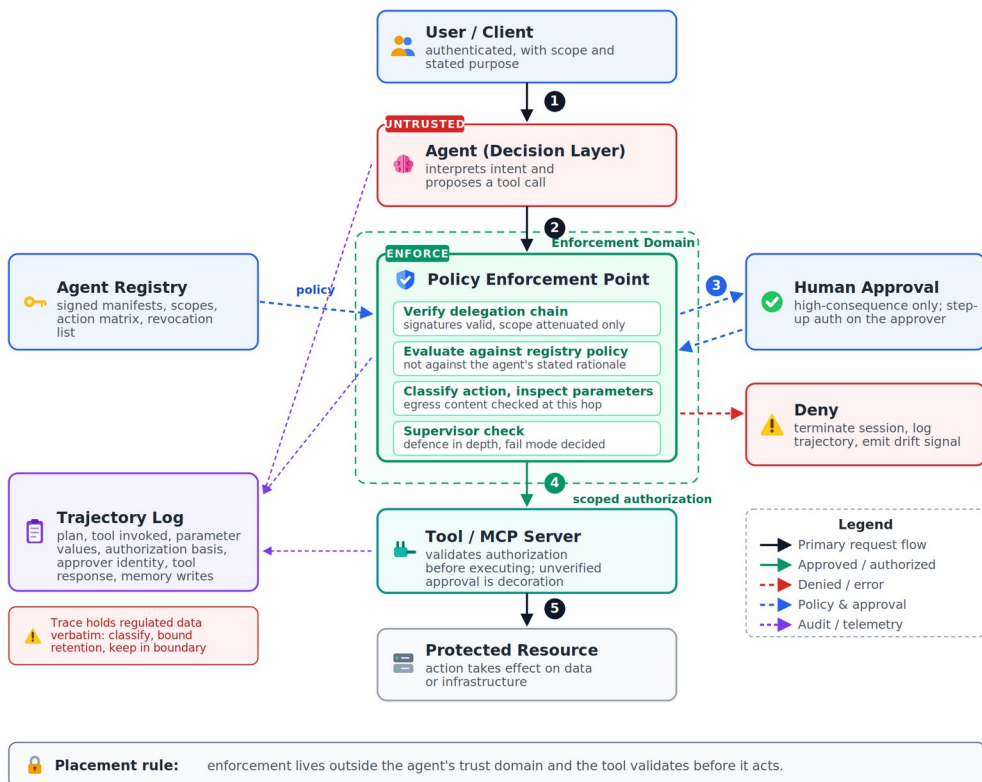


Figure 5. Authorization data flow through the enforcement point.

A signed approval that no downstream component verifies is decoration. When evaluating any product or design in this space, the first question is: *what component refuses the call, and is it reachable from the compromised path?*

4.3 Delegation across agent hops

Multi-agent systems require the originating principal's identity and authorization scope to travel with the request. Recommended pattern:

- **Token exchange (RFC 8693) or on-behalf-of flows** at each hop rather than the receiving agent acting under its own identity.
- **Attenuation only.** Each hop may narrow scope; no hop may broaden it. Privilege escalation across a delegation chain should be structurally impossible, not policy-prohibited.
- **Full chain in the authorization decision.** The PEP evaluates the entire delegation path, not just the immediate caller.
- **Purpose binding.** Where regulation requires a stated purpose for access (healthcare treatment/payment/operations, criminal justice purpose codes, financial-privacy provisions), the purpose is part of the delegated context and is evaluated at the tool boundary.

Delegation Across Agent Hops

Scope may narrow at each hop and may never widen. Escalation becomes structurally impossible, not policy-prohibited.

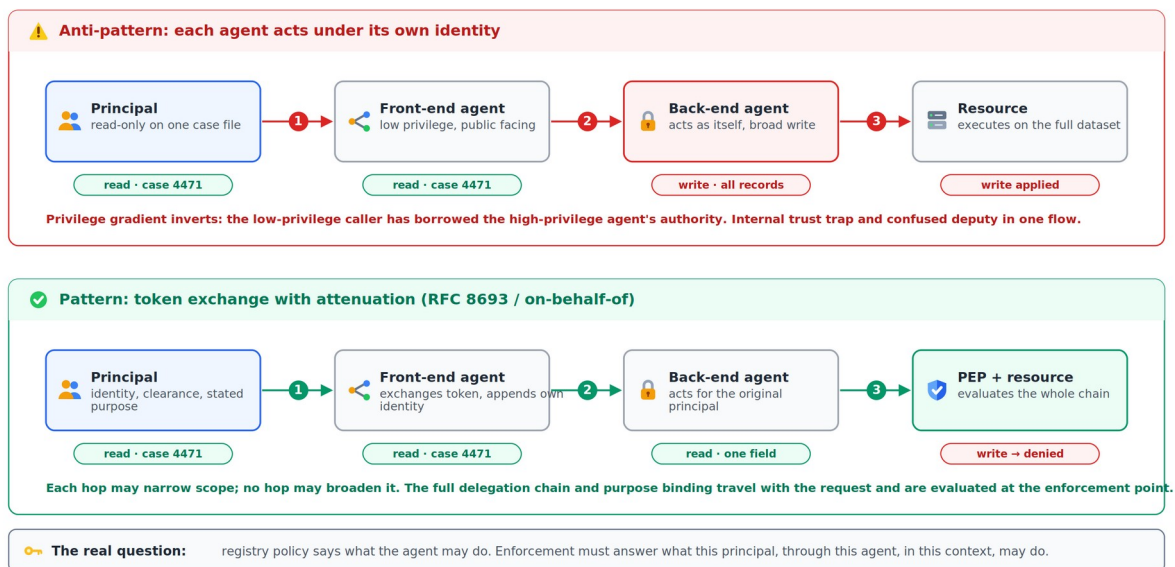


Figure 6. Delegation across agent hops with attenuation.

Without this, the internal trust trap and confused deputy remain open regardless of how well individual agents are governed.

4.4 Supervisory validation, and its limits

A secondary model reviewing proposed actions before execution is a useful defense-in-depth layer. It is **not** a primary control, for reasons that should be stated plainly in any design that uses it:

- The supervisor reads the primary agent's plan and rationale, text already shaped by whatever influenced the primary agent. Shared-compromise paths exist.

- It is a probabilistic classifier with a real false-negative rate. That rate must be measured and published internally, not assumed.
- It adds inference latency to every action, which matters in time-sensitive paths.
- Its failure mode must be an explicit decision. **Fail-closed** makes the supervisor a single point of denial for the workflow it protects. **Fail-open** makes it absent exactly when the system is under stress. Neither is wrong; leaving it undecided is.

Use the supervisor to catch plan/policy divergence that deterministic rules cannot express. Do not let its presence justify broader standing permissions.

4.5 Containment

- **Ephemeral, stateless execution sandboxes** (gVisor, Firecracker, or equivalent micro-VMs) for all agent code execution and tool interaction.
- **Per-connector process isolation.** A compromised ticketing connector must not be able to reach the database connector’s credentials or memory.
- **Zero-egress by default**, with dynamic allow-lists sourced from the registry per workflow.

Caveat that most guidance omits: network egress control does not cover **tool-mediated egress**. An agent that writes sensitive data into an allow-listed SaaS record, an approved ticket, or a permitted outbound message has exfiltrated it without a single packet leaving the sandbox to an unapproved destination. Egress must be controlled at the tool boundary, content inspection on outbound parameters, not only at the network layer.

4.6 The action risk matrix

Autonomy should be granted by action class and context, not globally. A workable matrix distinguishes four classes and separates the agent’s authorization from the approver’s authentication:

Action class	Risk driver	Control
Internal retrieval	Sensitivity and volume of data returned	Scoped read identity; volume and rate anomaly detection; full query logging
Disclosure / egress	Data crossing a trust or regulatory boundary	Content inspection on outbound parameters; classification-aware DLP at the tool boundary; approval required above defined sensitivity
State change (reversible)	Production write with a defined rollback path	Policy check at PEP; transaction logging; automated rollback capability verified
State change (irreversible or high-consequence)	Deletion, financial movement, safety-affecting action, privilege grant	Human approval with step-up authentication of the approver ; authorization verified at tool boundary before execution

Two corrections to the commonly published version of this table:

Disclosure is not a low-risk class. Nearly every threat in §3 terminates in either a state change or data egress. Matrices that gate retrieval heavily while treating outbound communication as low-risk have inverted the ranking relative to the actual threat model.

MFA does not apply to a non-human identity. Step-up authentication belongs to the *human approver* at the approval step. The agent presents a scoped credential; the human presents a second factor. Conflating these produces a control that cannot be implemented as written and reads as a category error to any competent assessor.

4.7 Designing human-in-the-loop that works

HITL is the most-recommended and least-analyzed control in this field. Its failure modes are predictable:

- **Approval fatigue.** Approval quality degrades with volume. If a design generates more than a few approvals per operator per hour, it produces rubber-stamping, not oversight. Estimate the volume during design.
- **Comprehension gap.** An operator asked to approve a multi-step plan they did not author, under time pressure, in a domain they may not specialize in, is not meaningfully reviewing it. Approval interfaces must surface the *effect*, what will change, to what, irreversibly or not, rather than the reasoning trace.
- **Accountability transfer.** HITL frequently functions to relocate liability onto an operator without giving them the information or time to exercise judgment. If the approver cannot realistically refuse, the control is theater.

Design target: few approvals, high consequence, effect-oriented presentation, and an explicit refusal path with no operational penalty.

5. Observability

5.1 Trajectory logging

Conventional logs (timestamp, principal, endpoint) are insufficient. Agentic audit requires the full trajectory: the plan, each tool invocation with **parameter values**, each tool response, each error and the agent's interpretation of it, and each memory write.

Parameter-level logging is the part usually missed and the part that matters. "Agent called the export tool" is not an audit record; "agent called the export tool with this filter, returning this row count, to this destination" is.

5.2 Reasoning traces, with a caveat

Capturing the model's intermediate reasoning aids forensic reconstruction. Two constraints belong in any design that relies on it:

Faithfulness. A model's stated reasoning is not a reliable transcript of its computation, and under adversarial conditions the rationale itself can be manipulated. Reasoning traces are *corroborating* evidence.

Determinations about whether an incident was error or compromise should rest on tool-call telemetry, which reflects what actually happened.

Classification. Reasoning traces contain, verbatim, whatever sensitive data the agent handled, regulated personal data, health information, financial records, criminal justice information. The trace store is a new high-sensitivity data repository that must be classified, access-controlled, retention-bounded, and located inside the appropriate authorization boundary. This is frequently overlooked because "logs" are assumed to be low-sensitivity by default.

5.3 Behavioral baselines and revocation

- **Goal drift detection**, action sequences diverging from the registered workflow. Note that baselines derived from staging will not match production distribution; expect a tuning period and budget the false-positive triage.
 - **Revocation (kill switch)**. Every agent requires manual revocation that invalidates credentials and terminates active execution. Three questions determine whether it works:
 - **Measured time-to-revoke**. If tokens are bearer credentials validated offline, revocation is not instant, it is bounded below by token TTL unless introspection or a revocation list is in place. Measure it; do not assume it.
 - **Authorization to invoke**. Who can pull it, at 3 a.m., without a change ticket?
 - **Fallback**. Revoking an agent a business process now depends on is itself an availability action. The manual fallback must be documented and periodically exercised.
-

6. Registry and Supply Chain Controls

The registry is simultaneously identity provider, policy store, and trust anchor. That concentration demands explicit treatment:

- **Signing and verification**. Agent definitions and tool manifests are cryptographically signed; the enforcement point verifies signatures before execution. Unsigned or unverified components are programmatically blocked, not flagged.
 - **Key custody**. Signing keys in an HSM or managed KMS with separation of duties between registry operators and key holders.
 - **Registry threat model**. Document what registry compromise yields and what constrains it. A trust anchor with no compensating controls is a single point of total failure in an architecture that otherwise assumes breach.
 - **Third-party MCP servers are external interconnections**. They belong in the system inventory, require the same due diligence as any external dependency, and in authorized environments may trigger interconnection agreements and change notification.
 - **Tool manifest pinning**. Version and hash pinning with change review, tool descriptions are model-consumed instructions and must be treated as code, not documentation.
-

7. Evaluation and Assurance

Most published agentic security guidance names testing tools and stops. Tools without a stated methodology, metrics, and numeric gates do not produce assurance. This section proposes the minimum.

7.1 Evaluate trajectories, not outputs

If the trajectory is where risk lives, it is where scoring must occur. Metrics:

- Tool-selection precision and recall against workflow-appropriate tools
- Unnecessary or out-of-scope invocation rate
- Parameter correctness, particularly on scoping filters
- Plan validity and policy conformance
- Recovery behavior on tool error, timeout, and malformed response

Output-only scoring cannot distinguish a correct answer reached through an unauthorized path, which is precisely the failure mode of interest.

7.2 Account for non-determinism

- **Report pass^k, not pass@k.** For consequential workflows the gate is *all k trials pass*, not *at least one*. State k, temperature, and seed policy.
- **Report variance**, not means alone. A control with high run-to-run variance has not been demonstrated.
- **Pin model versions.** A hosted-model update invalidates prior results and derived baselines. Define the re-evaluation trigger and turnaround, and decide explicitly whether a vendor model change constitutes a change requiring re-authorization.

7.3 Adversarial testing that means something

- **Adaptive adversary.** The attack generator must observe rejections and iterate. A fixed corpus measures memorization.
- **Corpus rotation.** A static red-team set becomes a benchmark the system is tuned to pass.
- **Human red teaming alongside automation.** Automated frameworks reliably underestimate a motivated human.
- **Domain-specific harm taxonomy.** Generic jailbreak metrics do not map to enterprise consequence. Score against the actual harm classes, unauthorized disclosure, unauthorized state change, safety-affecting action, regulatory violation, with severity tiers.
- **Judge validation.** If scoring uses a model as judge, that judge is reading the same potentially adversarial trajectory it grades. Validate against human labels and report agreement.

7.4 Environment fidelity

Agents behave very differently when tools fail. Staging harnesses that always return clean responses test a system that does not exist in production. Induced tool failures, latency injection, and malformed responses belong in the suite.

7.5 Coverage gaps to close deliberately

- **Multi-agent composition.** Per-agent evaluation will not surface the internal trust trap. Delegation-chain scenarios must be explicit test cases.
- **Cross-session persistence.** Memory poisoning manifests later, in a different session. Evaluation must span session boundaries or it structurally cannot detect the class.

7.6 Continuous evaluation

Evaluation does not end at the promotion gate. Shadow-mode execution, canary deployment, and production trajectory sampling with periodic scoring are what convert a point-in-time assessment into an ongoing control.

Evaluation and Promotion Gates

Non-deterministic systems are not cleared by a passing run. Gates are numeric and set before evaluation begins.

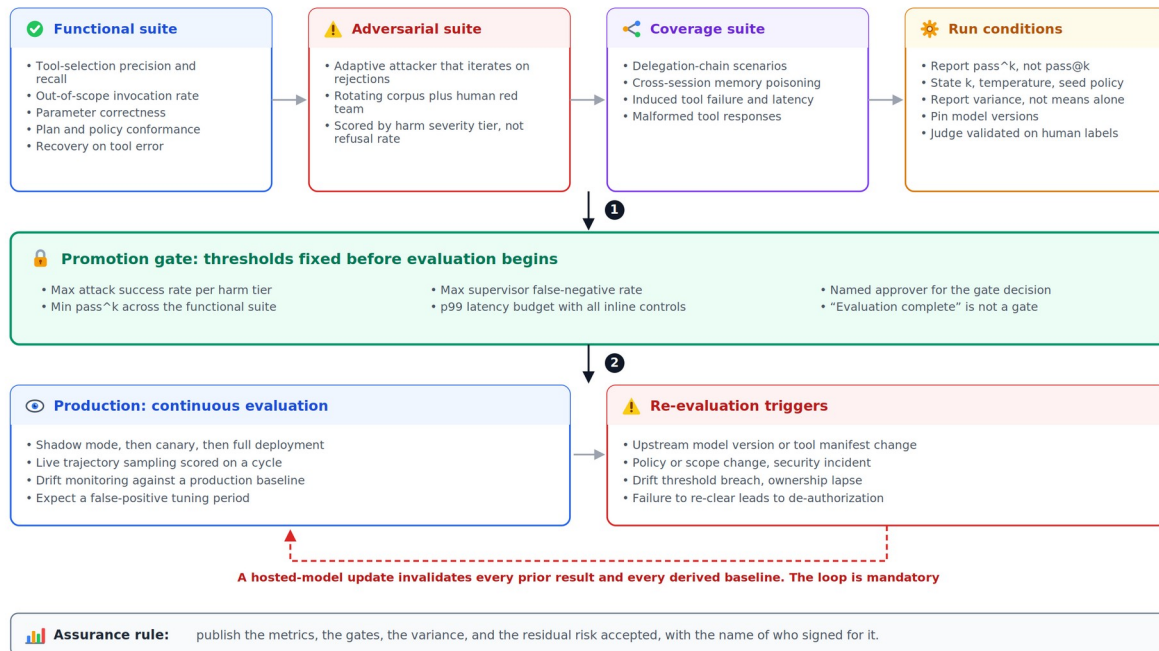


Figure 7. Evaluation suites, promotion gates, and re-evaluation triggers.

7.7 Promotion gates

Define numerically, before evaluation begins:

- Maximum acceptable attack success rate per harm severity tier
- Minimum pass[^]k across the functional suite
- Maximum acceptable supervisor false-negative rate
- Latency budget at p99 including all inline controls
- Named approver for the gate decision

"Evaluation complete" is not a gate.

8. Governance

8.1 Residual risk acceptance

Indirect prompt injection has no complete mitigation. Any architecture claiming to have solved it is misrepresenting the state of the field. A defensible governance posture therefore requires an explicit residual-risk statement: what is accepted, at what likelihood and impact, with what compensating controls, reviewed on what cadence, signed by a named accountable executive.

This is not a formality. It is the difference between an organization that has assessed agentic risk and one that has assumed it away.

8.2 Decommissioning and de-authorization

The kill switch is a mechanism; the policy for using it is separate and usually missing. Define:

- Criteria that trigger de-authorization (evaluation regression, incident, drift threshold, upstream model change, ownership lapse)
- Decision authority and escalation path
- Documented and exercised manual fallback for any process that has become agent-dependent
- Data disposition for memory stores and trajectory logs at decommission

8.3 Continuous authorization: the model that matches the artifact

Point-in-time authorization certifies a snapshot. An agentic system does not hold still for one: a hosted model updates, a tool manifest changes, a memory store accretes, and the system assessed last quarter no longer exists. This is the governance-tempo problem in its sharpest form, and the answer is not a faster annual cycle. It is continuous authorization (cATO), which this architecture supplies directly rather than by analogy:

- **The authorization decision** is the numeric promotion gate of Section 7.7: maximum attack success rate per harm tier, minimum pass^k, latency budget, named approver.
- **Continuous monitoring** is Section 7.6 in production: shadow and canary deployment, live trajectory sampling scored on a cycle, drift detection against a production baseline.
- **Re-authorization triggers** are the events of Section 8.2: model version change, manifest change, policy change, incident, drift breach. Failure to re-clear is de-authorization, automatically.
- **The body of evidence** is the parameter-level trajectory log of Section 5, generated by operation rather than assembled for an assessor.
- **Honest risk acceptance** is the signed residual-risk statement of Section 8.1, in place of checkbox artifacts that certify nothing.

Framed this way, the speed-versus-risk dilemma dissolves. An organization that builds enforcement and evaluation into the execution path does not choose between moving fast and staying authorized; the two become the same motion. Authorization stops being a gate the mission waits at and becomes a property the system carries.

8.4 Ownership

Every agent has a named technical owner and a named business owner. Ownership lapse is itself a de-authorization trigger. Orphaned agents with standing credentials are the agentic equivalent of orphaned service accounts, with more capability.

9. Control Crosswalk

Mapping the architecture to established control frameworks makes it usable in an authorization package rather than only as design guidance.

Architectural control	NIST SP 800-53 Rev 5	ISO/IEC 42001
Scoped non-human identity per agent	AC-2, IA-2, IA-5, IA-9	A.6 (AI system lifecycle), A.9 (Use of AI systems)
Least agency / dynamic tool scoping	AC-3, AC-6, CM-7	A.6, A.9

Architectural control	NIST SP 800-53 Rev 5	ISO/IEC 42001
Authorization enforced at tool boundary	AC-3, AC-4, AC-25	A.6
Delegation with attenuation	AC-3, AC-16, IA-9	A.6
Human approval for high-consequence actions	AC-3(2), AC-6(9)	A.9 (human oversight)
Trajectory and parameter-level logging	AU-2, AU-3, AU-12	A.5 (impact assessment), A.6
Trace store classification and protection	AU-9, MP-4, SC-28	A.7 (data for AI systems)
Goal drift / behavioral monitoring	SI-4, SI-4(2)	A.6 (monitoring)
Sandbox isolation	SC-7, SC-39, SC-44	A.6
Egress control (network and tool-mediated)	SC-7, AC-4, SI-4(18)	A.7
Registry signing and manifest verification	CM-14, SI-7, SA-10	A.10 (third-party)
Third-party MCP due diligence	SA-9, SR-3, SR-5, SR-11	A.10
Adversarial evaluation before promotion	CA-8, RA-5, SA-11	A.5, A.6
Residual risk acceptance	RA-3, PM-9	A.5
Revocation and decommissioning	AC-2(3), CM-3, SA-22	A.6

Organizations operating under sector-specific regimes should extend this crosswalk to their own obligations, and should treat the question of whether a non-human identity may access regulated data under delegated human authority as an **open item requiring a regulator or authorizing-official answer**, not an architectural assumption. Several major frameworks require unique identification of the individual accessing protected data, and delegation through an agent is not clearly addressed in current published guidance.

10. Implementation Roadmap

Phase 0: Inventory. Usually skipped, and skipping it invalidates everything after. Enumerate every agent, MCP server, and tool integration currently in production and pre-production; identify owners; identify the credentials each holds and the systems each reaches. Most organizations discover meaningful shadow deployment here. Without this, later phases have no scope and no denominator.

Phase 1: Interim controls and architecture decisions. Agents already in production do not pause for a roadmap. Apply immediate compensating controls, credential rotation to short-lived tokens, removal of standing write access, logging enablement, while making the architectural decisions that tooling cannot make: where the PEP lives, the delegation model, the action risk matrix, the registry design. These are build decisions, not buy decisions, and they must precede product selection.

Phase 2: Tooling selection against defined criteria. Evaluate candidates against the architecture, not the feature matrix. Key questions: Does it *enforce inline*, or only detect? Does it parse tool-protocol semantics, or only see TLS-terminated HTTP? What are its measured FPR/FNR against a corpus you control, not the vendor's demo set? What p99 latency does it add? Is it deployable in the boundary where the data actually lives? Are policies portable, or does the policy language create lock-in before the architecture is settled?

Note that posture management and traffic monitoring products are **detective**. If none of the candidates block an unauthorized tool call inline, the enforcement point is a build, and that belongs on the plan explicitly.

Phase 3: Staged deployment with gates. Per-agent promotion against the numeric gates in §7.7, with named approvers and documented artifacts.

Phase 4: Operate and sustain. Continuous evaluation, drift monitoring, corpus rotation, re-evaluation on model version change, periodic residual-risk review, and exercised fallback procedures. Labeling this phase “review” implies security is a terminal stage; it is the longest phase and the one where the posture is actually maintained.

Conclusion

The transition from conversational AI to agentic AI moves the security problem from the model’s output to the system’s execution path. The controls that matter are consequently not model controls. They are identity, authorization, delegation, isolation, and audit, the same disciplines that secure any distributed system, applied to a component that is non-deterministic, semantically attackable, and privileged.

Three principles carry most of the weight:

Least agency is the new least privilege. Constrain what actions exist, not only what data is visible. Bind tools to registered workflows, scope credentials per session, and separate read from write at the identity layer.

Enforcement belongs outside the agent. Any check the agent’s compromise can bypass is advisory. Authorization is verified at the tool boundary, against policy the agent cannot influence, by a component in a different trust domain.

Assurance requires stated methodology and accepted residual risk. Non-deterministic systems cannot be proven safe by a passing test run. Publish the metrics, the gates, the variance, and the risk you are accepting, and who signed for it.

The organizations that will deploy agentic AI successfully in regulated environments are not the ones that move fastest, and they are not the ones that refuse. They are the ones that bound the authorization blast radius precisely enough to let autonomy operate inside it, and instrument it well enough that the authorization never goes stale. That is what governing what the system does, not what it says, looks like in practice.

References

- U.S. Department of Defense CIO. *Memorandum: Continuous Authorization To Operate (cATO)*, 2022.
 - International Organization for Standardization / IEC. *ISO/IEC 42001: Information Technology, Artificial Intelligence, Management System*.
 - National Institute of Standards and Technology. *SP 800-53 Rev. 5, Security and Privacy Controls for Information Systems and Organizations*.
 - National Institute of Standards and Technology. *AI Risk Management Framework (AI RMF 1.0) and Generative AI Profile (NIST AI 600-1)*.
 - Cloud Security Alliance. *Security guidance for agentic workflows and non-human identity in cloud environments*.
 - OWASP Foundation. *Top 10 for LLM Applications and Agentic Security Initiative* threat taxonomy. <https://genai.owasp.org/>, confirm the current published version and identifier scheme before citing specific threat IDs.
 - Anthropic. *Model Context Protocol specification*. <https://modelcontextprotocol.io>
 - IETF. *RFC 8693, OAuth 2.0 Token Exchange*.
 - Google. *gVisor application kernel for container isolation*. <https://gvisor.dev/docs/>
 - Amazon Web Services. *Firecracker micro-VM*. <https://github.com/firecracker-microvm/firecracker>
 - Microsoft. *PyRIT, Python Risk Identification Tool for generative AI*. <https://github.com/Azure/PyRIT>
 - Open Container Initiative. *Runtime Specification*.
-

*Prepared by Optimal, LLC: AI security assessment, adversarial testing, and governed AI engineering.
gooptimal.io • ryan@gooptimal.io*